A CASE-STUDY BASED REPORT

ON

Waste Classification for Recycling Using CNN and Transfer Learning

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Keerthana.G **Roll No: 2520090132**

Sutte Alekhya **Roll No: 2520090126**

Under the Esteemed guidance of

Mr. Mekala Sanjeev Kumar

Asst.Professor, Dept. of CS&IT

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract 3

Introduction 4

Background 4
Motivation for the Project 4
Overview of the Work Carried Out 4
Scope and Responsible-Use Statement 4

Problem Statement 6

Case-Study Scenario 6
Central Case-Study Question 6
Formal Problem Definition 6
Objectives that Guide the Study 6
Assumptions and Limitations 6

Dataset and Exploratory Data Analysis 8

About the Dataset 8
Purpose of EDA 8
Preparation Steps Actually Performed 8
EDA Exploration - Implementation 9
Summary of EDA Findings 10

Algorithm Selection and Implementation 11

Conclusion 17

ABSTRACT

Improper segregation of municipal solid waste reduces the quantity of material that can be recycled and increases the load on landfills. Manual sorting is slow, costly and dependent on human judgment, so recyclable items such as plastic, paper, glass and metal are frequently lost when they are mixed with organic waste. This case study presents the design and implementation of an automated waste classification system that identifies the category of a waste item from a single photograph.

The system uses a Convolutional Neural Network (CNN) trained with Transfer Learning. Three pre-trained architectures, MobileNetV2, ResNet50 and EfficientNetB0, are adapted by replacing their final layers with a seven-class softmax classifier for the categories Plastic, Paper, Cardboard, Glass, Metal, Organic Waste and Trash. The models are trained on the Garbage Classification / TrashNet style dataset after resizing, normalisation, augmentation and a train, validation and test split. The trained model is served through a Python Flask REST API (POST /predict) and used from a responsive HTML5, CSS3 and JavaScript web interface where the user uploads or drops a photograph, previews it and receives the predicted class along with a confidence score. An optional SQLite module stores the prediction history (timestamp, category and confidence).

The work shows that combining transfer learning with a simple web front end makes waste identification possible in a single click, reduces the dependence on manual segregation and supports faster, more reliable recycling.

Keywords: Waste Classification, Convolutional Neural Network, Transfer Learning, MobileNetV2, ResNet50, EfficientNetB0, Flask, Recycling.

1. Introduction

1.1 Background

Rapid urbanisation and changing consumption patterns have increased the volume and variety of municipal solid waste. Recycling is one of the most effective ways to reduce this burden, but it only works when waste is separated correctly at the source or at the sorting facility. In most places this separation is still performed by hand. Workers must recognise the material of every item, which is slow, tiring and error-prone, and mixed or contaminated waste is often sent to landfill even when it is recyclable.

Computer vision gives a way to automate this recognition. Deep learning models, and CNNs in particular, learn visual features such as edges, shapes and textures directly from images and have become the standard approach for image classification. Transfer learning further allows models that were trained on very large image collections to be reused for a new task with a comparatively small dataset, which suits waste classification where labelled data is limited.

1.2 Motivation for the Project

- **Slow and costly:** manual segregation is labour-intensive and time-consuming.
- **Error-prone:** sorting depends on human judgment and often results in misclassification.
- **Lost recyclables:** plastic, paper, glass and metal mixed with organic waste are harder to recover.
- **Accessibility:** an intelligent system should be usable by anyone through a simple web interface, without special hardware or training.

1.3 Overview of the Work Carried Out

The project covers the complete path from data to a usable application:

1. Identification of the Garbage Classification / TrashNet style dataset organised into seven labelled class folders.
2. Preprocessing of the images (resizing to 224 × 224, RGB conversion, normalisation, augmentation) and splitting into training, validation and test sets.
3. Training of CNN classifiers using transfer learning with MobileNetV2, ResNet50 and EfficientNetB0, and saving the trained model (.h5 / .keras).
4. Development of a Flask REST API that preprocesses an uploaded image and returns the predicted class and confidence as JSON.
5. Development of a responsive web front end for image upload, preview, classification and result display.
6. An optional SQLite module that records the prediction history.

1.4 Scope and Responsible-Use Statement

The scope of the study is image-based classification of a single waste item into one of seven categories using a web application. The system is intended as a decision-support and awareness tool for learning and for assisting segregation; it is not a certified sorting system. Predictions are accompanied by a confidence score so that users can judge how reliable a result is, and low-

confidence predictions should be checked manually. Hazardous, medical or electronic waste is outside the scope, and the system must not be relied upon to decide how such items are handled. Uploaded images are used only for prediction, and any stored history contains only the timestamp, category and confidence, not personal data.

2. Problem Statement

2.1 Case-Study Scenario

A residential community and a small recycling centre receive mixed household waste every day. Staff separate plastic bottles, paper, cardboard, glass, metal cans and food waste by hand. The process is slow, different workers make different decisions about the same item, and recyclable material that is contaminated by organic waste ends up in landfill. The community wants a low-cost tool that anyone can use from a phone or a computer: take a photograph of an item, and be told what type of waste it is.

2.2 Central Case-Study Question

Can a CNN trained with transfer learning classify a photograph of a waste item into one of seven categories accurately enough to be offered, through a simple web interface, as an automatic aid for waste segregation and recycling?

2.3 Formal Problem Definition

Let X be the set of RGB images of waste items and Y = {Plastic, Paper, Cardboard, Glass, Metal, Organic Waste, Trash} the set of seven class labels. Given a labelled dataset D = {(x_i, y_i)}, the aim is to learn a function f: X → Y, parameterised by a CNN with weights θ, that minimises the categorical cross-entropy loss

$$L(\theta) = -\Sigma_i \Sigma_c \; y_{ic} \cdot \log p_\theta(c \mid x_i)$$

where p_θ(c | x) is the softmax probability of class c. At inference time, the predicted class is $\hat{y}$ = argmax_c p_θ(c | x) and the associated confidence is max_c p_θ(c | x). The learned model is to be deployed behind a REST API so that a front-end client can obtain ($\hat{y}$, confidence) for any uploaded image.

2.4 Objectives that Guide the Study

1. To build an image dataset pipeline for seven waste classes with resizing, normalisation, augmentation and a train, validation and test split.
2. To train and compare CNN classifiers based on MobileNetV2, ResNet50 and EfficientNetB0 using transfer learning.
3. To evaluate the models using accuracy, precision, recall, F1-score and the confusion matrix on unseen test data.
4. To expose the trained model through a Flask REST API with a /predict endpoint.
5. To develop a user-friendly web interface that uploads an image, previews it and displays the predicted category with its confidence.
6. To optionally record the prediction history (timestamp, category, confidence) in a SQLite database.

2.5 Assumptions and Limitations

Assumptions

- Each image contains one dominant waste item that is clearly visible.

- The dataset labels are correct and the seven classes cover the items users will upload.
- Images are ordinary RGB photographs; the model works at an input size of 224 × 224.
- Pre-trained ImageNet features are useful for waste images.

Limitations

- The model can only predict the seven trained classes; an unknown item will still be assigned to one of them.
- Datasets of the TrashNet type are captured on plain backgrounds, so accuracy can fall on cluttered, poorly lit or partly hidden items in real photographs.
- Visually similar classes (for example paper and cardboard, or glass and plastic) are more likely to be confused, and the class “Trash” is typically under-represented.
- A photograph shows a single item; mixed piles and multi-object scenes are not handled.
- The prediction speed depends on the server hardware, and the optional database is not intended for large-scale use.

3. Dataset and Exploratory Data Analysis

3.1 About the Dataset

The project uses the Garbage Classification / TrashNet style dataset. The images are organised in seven labelled class folders, one folder per waste category:

Class	Typical items
Plastic	Bottles, containers, wrappers
Paper	Sheets, newspapers, crumpled paper
Cardboard	Boxes and packaging
Glass	Bottles and jars
Metal	Cans, tins, metal pieces
Organic Waste	Food and vegetable waste
Trash	Non-recyclable general waste

Table 1: Waste classes used in the dataset

The folder name serves as the class label. After resizing and normalisation the data are divided into training, validation and test sets.

3.2 Purpose of EDA

Exploratory Data Analysis was carried out before training to understand the data and to prevent avoidable modelling problems. It was used to (i) check how many images each class contains and whether the classes are balanced, (ii) confirm that all files are valid images and examine their sizes and colour channels, (iii) look at sample images of each class to understand within-class variation and between-class similarity, and (iv) decide which preprocessing and augmentation steps are needed.

3.3 Preparation Steps Actually Performed

1. **Loading and labelling:** images were read class by class from the seven folders and the folder name was used as the label.
2. **Invalid-image check:** unreadable or corrupted files were detected and removed.
3. **Resizing:** every image was resized to 224 × 224 pixels, the input size of the pre-trained networks.
4. **RGB conversion:** all images were converted to three-channel RGB.
5. **Normalisation:** pixel values were scaled (to the 0–1 range or with the preprocessing function of the chosen network).
6. **Data augmentation:** rotation, flipping, zooming and shifting were applied to the training images to increase variety and reduce over-fitting.
7. **Train / validation / test split:** the data were divided into training, validation and test subsets, with the test set kept untouched until the final evaluation.

3.4 EDA Exploration – Implementation

The following code counts the images in each class, checks that every file can be opened, and displays a few samples from each class.

```
import os, random
import matplotlib.pyplot as plt
from PIL import Image

DATA_DIR = "dataset"                        # one sub-folder per class
classes  = sorted(os.listdir(DATA_DIR))

# 1. Class distribution
counts = {c: len(os.listdir(os.path.join(DATA_DIR, c))) for c in classes}
plt.figure(figsize=(8, 4))
plt.bar(counts.keys(), counts.values())
plt.title("Number of images per class")
plt.xticks(rotation=30); plt.ylabel("Images"); plt.tight_layout(); plt.show()

# 2. Corrupted-file and image-size check
bad, sizes = [], []
for c in classes:
    for f in os.listdir(os.path.join(DATA_DIR, c)):
        path = os.path.join(DATA_DIR, c, f)
        try:
            with Image.open(path) as im:
                im.verify()
            sizes.append(Image.open(path).size)
        except Exception:
            bad.append(path)
print("Corrupted files:", len(bad), "| distinct image sizes:", len(set(sizes)))

# 3. Sample images from every class
fig, ax = plt.subplots(len(classes), 4, figsize=(8, 2 * len(classes)))
for i, c in enumerate(classes):
    files = random.sample(os.listdir(os.path.join(DATA_DIR, c)), 4)
    for j, f in enumerate(files):
        ax[i, j].imshow(Image.open(os.path.join(DATA_DIR, c, f)))
        ax[i, j].axis("off")
    ax[i, 0].set_title(c, loc="left", fontsize=9)
plt.tight_layout(); plt.show()
```

Data loading, augmentation and splitting used the Keras image utilities:

```
import tensorflow as tf
IMG_SIZE, BATCH = (224, 224), 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    "dataset", validation_split=0.30, subset="training",
    seed=42, image_size=IMG_SIZE, batch_size=BATCH, label_mode="categorical")
tmp_ds = tf.keras.utils.image_dataset_from_directory(
    "dataset", validation_split=0.30, subset="validation",
    seed=42, image_size=IMG_SIZE, batch_size=BATCH, label_mode="categorical")
val_ds  = tmp_ds.take(len(tmp_ds) // 2)      # validation set
test_ds = tmp_ds.skip(len(tmp_ds) // 2)      # held-out test set

augment = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.15),
    tf.keras.layers.RandomZoom(0.15),
    tf.keras.layers.RandomTranslation(0.1, 0.1),
])
```

[Insert here the class-distribution bar chart and the sample-image grid generated by the code above.]

3.5 Summary of EDA Findings

- **Seven well-defined classes:** the dataset is organised in seven folders (Plastic, Paper, Cardboard, Glass, Metal, Organic Waste, Trash), so it maps directly to a seven-neuron softmax output.
- **Possible class imbalance:** classes in datasets of this type are not equally sized (the "Trash" class is usually the smallest). The per-class counts should be checked, and augmentation (and, if required, class weights) is used to reduce its effect.
- **Variable image sizes:** the original images are not all the same size, so they must be resized to 224 × 224 before being given to the pre-trained networks.
- **Visual similarity between classes:** paper and cardboard, and glass and plastic, share colours and textures and are the most likely to be confused; this is why the confusion matrix is examined during evaluation.
- **Invalid files:** corrupted or unreadable images, if present, are removed in the invalid-image check so that they do not stop training.
- **Need for augmentation:** variation in angle, lighting and position justifies rotation, flip, zoom and shift augmentation to improve generalisation.

4. Algorithm Selection and Implementation

4.1 Why a CNN?

A Convolutional Neural Network is the natural choice for image classification. Its convolution layers apply learnable filters that detect local patterns, ReLU activations add non-linearity, pooling layers reduce the spatial size while keeping the important features, and fully connected layers combine the features for classification. Early layers learn edges, deeper layers learn shapes, textures and object parts, which is exactly the information needed to tell a plastic bottle from a glass bottle or a cardboard box. Features are learned automatically, so no manual feature engineering is required.

4.2 Why Transfer Learning?

Training a deep CNN from scratch needs a very large dataset and a lot of computation. Transfer learning reuses a network that has already been trained on ImageNet: its convolutional base (the feature extractor) is kept, a new classification head is attached, and only the head is trained first. Afterwards some of the top layers of the base can be unfrozen and fine-tuned with a small learning rate. This reuses the learned image features, reduces the training time and gives better accuracy on a small dataset.

4.3 Pre-trained Models Used

Model	Key idea	Approx. parameters	Input
MobileNetV2	Depth-wise separable convolutions with inverted residual blocks; light and fast	≈ 3.4 M	224×224×3
ResNet50	50-layer network with residual (skip) connections that ease the training of deep models	≈ 25.6 M	224×224×3
EfficientNetB0	Compound scaling of depth, width and resolution; high accuracy for its size	≈ 5.3 M	224×224×3

Table 2: Pre-trained backbones compared in the project (parameter counts for the ImageNet versions without the top layer are slightly lower)

MobileNetV2 is the best fit when speed and a small model are important, for example for deployment on a phone; ResNet50 is a strong, widely used baseline; EfficientNetB0 offers a good accuracy-to-size balance. Training all three allows the final model to be chosen on the basis of the test results rather than assumption.

4.4 Model Architecture

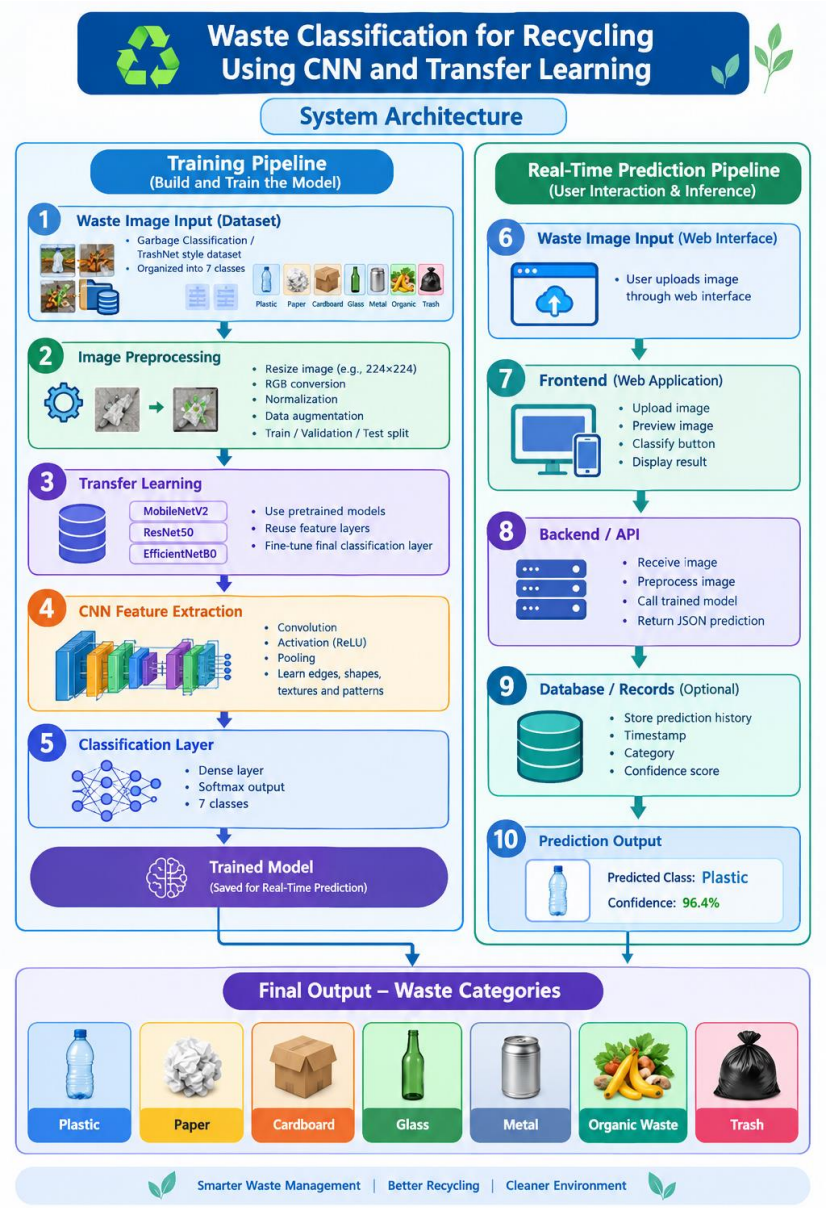


Figure 1: System architecture – training pipeline (left) and real-time prediction pipeline (right)

Training pipeline: dataset → preprocessing → transfer learning (MobileNetV2, ResNet50, EfficientNetB0) → CNN feature extraction → seven-class softmax → saved model.

Real-time prediction pipeline: web interface → frontend → backend API → trained model → optional database → predicted class with confidence.

4.5 Implementation of the Model

```
from tensorflow.keras import layers, models, optimizers, callbacks
from tensorflow.keras.applications import MobileNetV2, ResNet50, EfficientNetB0
from tensorflow.keras.applications import mobilenet_v2, resnet, efficientnet

NUM_CLASSES = 7
BACKBONES = {
    "mobilenetv2":    (MobileNetV2,    mobilenet_v2.preprocess_input),
    "resnet50":       (ResNet50,       resnet.preprocess_input),
```

```
    "efficientnetb0": (EfficientNetB0, efficientnet.preprocess_input),
}

def build_model(name):
    Base, preprocess = BACKBONES[name]
    base = Base(weights="imagenet", include_top=False, input_shape=(224, 224, 3))
    base.trainable = False                          # stage 1: feature extraction

    inputs  = layers.Input(shape=(224, 224, 3))
    x = augment(inputs)                             # augmentation (training only)
    x = preprocess(x)                               # model-specific normalisation
    x = base(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(128, activation="relu")(x)
    outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)
    return models.Model(inputs, outputs), base

model, base = build_model("mobilenetv2")
model.compile(optimizer=optimizers.Adam(1e-3),
              loss="categorical_crossentropy", metrics=["accuracy"])

cb = [callbacks.EarlyStopping(patience=4, restore_best_weights=True),
      callbacks.ReduceLROnPlateau(patience=2, factor=0.3)]
model.fit(train_ds, validation_data=val_ds, epochs=15, callbacks=cb)

# stage 2: fine-tune the top of the backbone with a small learning rate
base.trainable = True
for layer in base.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=optimizers.Adam(1e-5),
              loss="categorical_crossentropy", metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=10, callbacks=cb)

model.save("waste_classifier.keras")
```

The hyper-parameters shown (learning rates, dropout, number of fine-tuned layers, epochs) are typical settings; the same function is called with "resnet50" and "efficientnetb0" to train the other two models.

4.6 Model Evaluation

The models are evaluated on the unseen test set with the following measures:

- **Accuracy:** fraction of test images classified correctly.
- **Precision, recall and F1-score** for every class, which show how well each category is recognised.
- **Confusion matrix:** shows which classes are confused with each other.

```
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix

y_true, y_pred = [], []
for x, y in test_ds:
    y_true += list(np.argmax(y.numpy(), axis=1))
    y_pred += list(np.argmax(model.predict(x, verbose=0), axis=1))

print(classification_report(y_true, y_pred, target_names=class_names))
print(confusion_matrix(y_true, y_pred))
```

[Insert the accuracy / precision / recall / F1 table, the training curves and the confusion matrix obtained from your own run, and state which of the three models gave the best result.]

4.7 Backend: Prediction API

The backend is a Python Flask application. It receives the image sent by the front end, performs the same preprocessing used in training (resize to 224 × 224, RGB, normalisation), runs the trained model and returns the result as JSON.

Step	Action
1. Receive	Accept the image through POST /predict
2. Preprocess	Resize to 224 × 224, convert to RGB, normalise
3. Predict	Load the trained CNN and run inference
4. Respond	Return JSON containing class and confidence

Table 3: Steps performed by the prediction API

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from PIL import Image
import numpy as np, tensorflow as tf, sqlite3, datetime

app = Flask(__name__); CORS(app)
model = tf.keras.models.load_model("waste_classifier.keras")
CLASSES = ["Cardboard", "Glass", "Metal", "Organic Waste", "Paper", "Plastic", "Trash"]

def preprocess(file):
    img = Image.open(file.stream).convert("RGB").resize((224, 224))
    return np.expand_dims(np.array(img, dtype="float32"), axis=0)

@app.route("/predict", methods=["POST"])
def predict():
    if "file" not in request.files:
        return jsonify({"error": "No image uploaded"}), 400
    probs = model.predict(preprocess(request.files["file"]), verbose=0)[0]
    idx = int(np.argmax(probs))
    result = {"class": CLASSES[idx], "confidence": round(float(probs[idx]), 3)}

    # optional: store the prediction history
    with sqlite3.connect("history.db") as db:
        db.execute("CREATE TABLE IF NOT EXISTS history"
                   "(ts TEXT, category TEXT, confidence REAL)")
        db.execute("INSERT INTO history VALUES (?,?,?)",
                   (datetime.datetime.now().isoformat(), result["class"],
result["confidence"]))
    return jsonify(result)

if __name__ == "__main__":
    app.run(debug=True)
```

Example request and response:

```
POST /predict
file: waste_image.jpg

200 OK
{"class": "Plastic", "confidence": 0.964}
```

The order of CLASSES must be identical to the class order used while training (alphabetical folder order in Keras), and the preprocessing in the API must match the model that was saved.

4.8 Frontend: Web Application

The frontend is a responsive web page written in HTML5, CSS3 and JavaScript that calls the backend through the Fetch API. It provides:

- **Image upload:** browse or drag-and-drop a waste photograph.
- **Live preview:** the user confirms the image before sending it.
- **Classify button:** sends the image to the backend API.
- **Result display:** shows the predicted class and the confidence, e.g. “Predicted: Plastic Confidence: 96.4%”.

```
const form = document.getElementById("uploadForm");
form.addEventListener("submit", async (e) => {
  e.preventDefault();
  const data = new FormData();
  data.append("file", document.getElementById("imageInput").files[0]);

  const res  = await fetch("http://127.0.0.1:5000/predict", { method: "POST", body: data
});
  const out  = await res.json();
  document.getElementById("result").innerText =
      `Predicted: ${out.class}   Confidence: ${(out.confidence * 100).toFixed(1)}%`;
});
```

4.9 End-to-End Workflow

Stage	Actor	Action
1	User	Selects an image
2	Frontend	Previews the image and sends it to the API
3	Backend	Preprocesses the image
4	CNN model	Predicts the class
5	Frontend	Displays the class and the confidence

Table 4: End-to-end workflow of the system

The user therefore sees the waste category and its confidence within seconds, without any manual sorting.

4.10 Technology Stack

Layer	Technologies
Frontend	HTML5, CSS3, JavaScript; responsive web UI; Fetch API for REST calls
Backend	Python and Flask REST API; TensorFlow / Keras inference; Pillow and NumPy preprocessing
ML model	MobileNetV2, ResNet50, EfficientNetB0; saved model (.h5 / .keras); softmax over 7 classes

Storage (optional)	SQLite prediction history: timestamp, category, confidence

Table 5: Technology stack

5. Conclusion

This case study developed an automated waste classification system that identifies the category of a waste item from a photograph. A CNN based on transfer learning with MobileNetV2, ResNet50 and EfficientNetB0 was trained on a seven-class waste dataset and deployed through a Flask REST API and a browser-based front end that returns the predicted class and confidence within seconds.

- **Accurate model:** the CNN learns shapes, textures and patterns of waste items, and transfer learning gives good features with limited data and short training time.
- **Easy to use:** the web frontend and Flask backend make classification a single click for any user.
- **Greener recycling:** less manual segregation, faster sorting and better environmental management.

Future scope: the system can be extended with more waste categories, a larger and more varied real-world dataset, multi-object detection for mixed piles, a mobile application or an edge device with a lightweight model such as MobileNetV2, and a dashboard that analyses the stored prediction history.

References

1. M. Yang and G. Thung, "Classification of Trash for Recyclability Status," Stanford University, CS229 Project Report, 2016 (TrashNet).
2. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
3. K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016.
4. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019.
5. TensorFlow / Keras Applications documentation, https://keras.io/api/applications/
6. Flask documentation, https://flask.palletsprojects.com/

Signature of the Team members